Advanced SQL Assignment Questions

COURSE: DATA ANALYTICS

Part 1: Theoretical Concepts

Q1. Common Table Expression (CTE)

- **What it is:** A temporary named result set available only within the execution scope of a single query.
- **Readability:** It replaces messy nested subqueries with a sequential, clean, top-down structure.

Q2. Updatable vs Read-Only Views

- **Explanation:** Updatable views directly reference columns of a single base table (p. 2). Views become read-only if they use aggregations (***SUM, AVG***), ***GROUP BY***, or ***DISTINCT***, because the database cannot track which exact row to modify (p. 2).
- **Example:**
  - *Updatable:* ***SELECT ProductID, Price FROM Products*** (maps to direct rows) (p. 2).
  - *Read-Only:* ***SELECT Category, AVG(Price) FROM Products GROUP BY Category*** (cannot update an average) (p. 2).

Q3. Advantages of Stored Procedures

- **Performance:** Pre-compiled and optimized on the server, saving execution time.
- **Security:** Prevents SQL injection and allows restricting direct table access.
- **Maintenance:** Code is centralized; changes are made in one place instead of modifying multiple backend scripts.

Q4. Purpose of Triggers

- **Purpose:** Automatically execute a block of code in response to specific events like ***INSERT***, ***UPDATE***, or ***DELETE*** (p. 3).
- **Essential Use Case:** Automated data auditing (e.g., logging old salary amounts and who changed them whenever an employee record updates).

Q5. Data Modelling and Normalization

- **Data Modelling:** Creates a clear structural blueprint to ensure data accurately reflects business rules.
- **Normalization:** Eliminates redundant data to save storage space and prevent data update inconsistencies.

Part 2: Practical SQL Queries

Q6. Revenue Calculation CTE

```
WITH ProductRevenue AS (
    SELECT
        p.ProductID,
        p.ProductName,
        SUM(p.Price * s.Quantity) AS TotalRevenue
    FROM Products p
    JOIN Sales s ON p.ProductID = s.ProductID
    GROUP BY p.ProductID, p.ProductName
)
SELECT *
FROM ProductRevenue
WHERE TotalRevenue > 3000;
```

Q7. Category Summary View

```
CREATE VIEW vw_CategorySummary AS
SELECT
    Category,
    COUNT(ProductID) AS TotalProducts,
    AVG(Price) AS AveragePrice
FROM Products
GROUP BY Category;
```

Q8. Updatable View & Update Query

```
-- 1. Create the updatable view
CREATE VIEW vw_UpdatableProducts AS
SELECT ProductID, ProductName, Price
FROM Products;

-- 2. Update price using the view
UPDATE vw_UpdatableProducts
SET Price = 1300.00
WHERE ProductID = 1;
```

Q9. Category Stored Procedure

```
DELIMITER //
CREATE PROCEDURE GetProductsByCategory(IN cat_name VARCHAR(50))
BEGIN
    SELECT ProductID, ProductName, Category, Price
    FROM Products
    WHERE Category = cat_name;
END //
DELIMITER ;
```

Q10. AFTER DELETE Archive Trigger

```
-- 1. Create the archive table structure
CREATE TABLE ProductArchive (
    ProductID INT,
    ProductName VARCHAR(100),
    Category VARCHAR(50),
    Price DECIMAL(10,2),
    DeletedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Create the trigger
DELIMITER //
CREATE TRIGGER Before_Products_Delete
AFTER DELETE ON Products
FOR EACH ROW
BEGIN
    INSERT INTO ProductArchive (ProductID, ProductName, Category, Price)
    VALUES (OLD.ProductID, OLD.ProductName, OLD.Category, OLD.Price);
END //
DELIMITER ;
```